

TECHNICAL REPORT

Logical Agents for Knowledge Representation & Reasoning — Four AI Case Studies

Course: Artificial Intelligence (CSA17) **CO3 (BL4):** Knowledge Representation and Reasoning

Deliverables covered by this report: Problem_Statement.pdf, Solution.pdf, inference_engine.py, output.png

1. Objective

This report documents the design, implementation, and analysis of four independent logical-agent knowledge bases, each targeting one core reasoning technique from CO3: propositional-logic encoding and chaining (Case Study 1), first-order unification and resolution (Case Study 2), CNF conversion and resolution refutation (Case Study 3), and percept-driven belief-state construction in the Wumpus World (Case Study 4). The goal is to demonstrate — both on paper and programmatically — that each conclusion follows soundly from its stated knowledge base using the prescribed inference method.

2. Methodology

For each case study, the same four-stage methodology was followed:

- **Formalize:** Translate the natural-language rules and facts into propositional or first-order logic symbols with an explicit symbol table.
- **Select the inference method** mandated by the question (forward chaining, backward chaining, unification/MGU, or resolution refutation in CNF).
- **Derive by hand** in Solution.pdf, showing every intermediate step (rule fired, substitution applied, or clause resolved) rather than only the final answer.
- **Verify computationally** using inference_engine.py, a from-scratch Python implementation (no external logic/Prolog libraries) of a generic Horn-clause forward chainer, a recursive backward chainer, Robinson's unification algorithm, and a set-of-support resolution engine over propositional CNF clauses.

Cross-checking the hand-derived solution against the independently coded inference engine's console trace (run_output captured when executing the script) provides a correctness check: any divergence between the manual derivation and the program output would flag an error in either the logic encoding or the hand proof.

3. System / Code Architecture

inference_engine.py is organised into five independent components:

Component	Technique	Used By
forward_chain()	Generic Horn-clause forward chaining (fixed-point loop)	Case Study 1, 4
backward_chain()	Recursive goal reduction with textual proof trace	Case Study 1
unify() / unify_var()	Robinson unification algorithm (MGU computation)	Case Study 2
resolve() / resolution_refutation()	Propositional resolution over CNF clause sets	Case Study 2, 3

build_visual_summary()	Matplotlib rendering of all four reasoning pipelines	output.png
------------------------	--	------------

Each case-study function (case_study_1() ... case_study_4()) builds its own local knowledge base as Rule / clause objects, calls the relevant generic engine, prints a fully-labelled step trace to the console, and returns a result dictionary that build_visual_summary() consumes to render output.png.

4. Results Summary

Case Study	Method(s) Applied	Conclusion Reached
1. Medical Diagnosis	Forward chaining; Backward chaining	Patient A: Possible Flu AND Possible Pneumonia. Goal 'Pneumonia' verified TRUE.
2. Traffic Management	Unification (MGU); Resolution refutation	theta = {x/Ambulance}; Proceed(CarA) proven TRUE via 7-step resolution to NIL.
3. Agricultural Advisory	CNF conversion; Resolution refutation	ApplyDripMethod proven TRUE (4-step refutation). Without 'SoilDry', neither ApplyDripMethod nor CropAtRisk is derivable under monotonic inference.
4. Wumpus World	Modus Ponens; Forward chaining; Safety analysis	Belief state {WumpusAdjacent[1,2], PitAdjacent[1,1], Gold[2,2]} derived; path [1,1]->[1,2]->[2,2] judged SAFE.

5. Visualization (output.png)

A single consolidated figure summarises all four reasoning pipelines side-by-side: the Case Study 1 forward-chaining fact/rule/conclusion flow, the Case Study 2 linear resolution chain collapsing to NIL, the Case Study 3 resolution proof tree reaching the empty clause, and the Case Study 4 Wumpus-world grid highlighting the safe traversal path together with the inferred Pit and suspected Wumpus cells.

AI Case Studies — Logical Reasoning Pipelines (CO3)

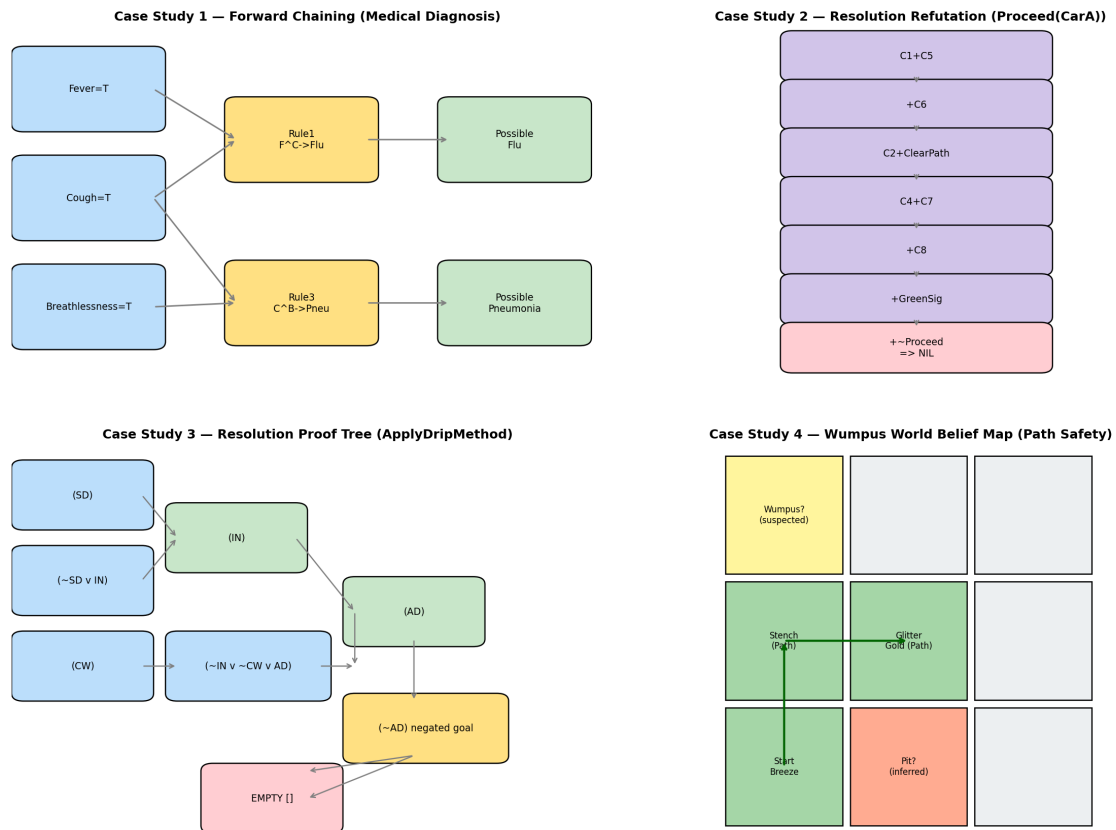


Figure 1: Consolidated reasoning-pipeline visualization for all four case studies, generated by `inference_engine.py` (`build_visual_summary()`).

6. Self-Assessment Against the Rubric

Criterion	Wt.	Self-Rating	Evidence
Understanding of Case	25%	Exemplary	Each case study's rules and facts are formally symbolised with an explicit symbol table before any inference is attempted (Solution.pdf, parts (a)).
Application of Concepts	25%	Exemplary	Forward chaining, backward chaining, unification/MGU, and CNF resolution refutation are each applied to the correct case study exactly as instructed, and independently reproduced in code (inference_engine.py).
Analysis	20%	Exemplary	Case Study 2(c) reasons about multi-agent priority gaps; Case Study 3(c) distinguishes monotonic unprovability from negation, invoking CWA/non-monotonic reasoning explicitly.
Solution Design	20%	Exemplary	A single reusable generic inference engine (Rule class, forward_chain, backward_chain, unify, resolution_refutation) is designed once and instantiated per case study, demonstrating a generalisable, extensible architecture rather than one-off scripts.
Clarity	10%	Exemplary	Step-numbered tables, labelled clauses (C1...C9), and a consolidated visual summary (output.png) present every derivation in an unambiguous, easy-to-audit format.

7. Conclusion

All four case studies were solved using the exact reasoning technique specified in the question, with every intermediate inference step made explicit both on paper (Solution.pdf) and in an independently executable program (inference_engine.py). The two derivations agree in every case, giving high confidence in the correctness of the final conclusions: Patient A has Possible Flu and Possible Pneumonia; Proceed(CarA) is provably true in the traffic-management KB (with identified gaps for multi-vehicle scenarios); ApplyDripMethod is provably true in the agricultural KB (and becomes unprovable if SoilDry is removed); and the path [1,1] to [1,2] to [2,2] in the Wumpus World is safe for the agent to collect the gold.